

DOMAIN-SPECIFIC RAG CHATBOT FOR PDF QUESTION ANSWERING

A Project Report Submitted as Part of the Unlox Academy Internship Program

Major Project - 3

Submitted by:

Navyashree K N
Master of Computer Applications (MCA)
Dayananda Sagar College of Arts, Science, And Commerce
Bangalore

Internship Details:

Organization: Unlox Academy
Track: Artificial Intelligence (AI)
Submission Date: 25-09-2026

1. Problem Statement

Large documents such as manuals, policy files, course notes, and interview preparation guides are often difficult to search manually. A user may need to read through many pages to locate a single piece of information. This project addresses that problem by allowing users to upload PDF documents and ask questions in natural language, receiving answers grounded strictly in the uploaded content, along with the exact source document and page number for verification.

2. Objective

The objective of this project is to build a reliable, domain-specific question-answering system using the Retrieval-Augmented Generation (RAG) approach. The system retrieves the most relevant passages from user-uploaded PDF documents and generates answers based only on that retrieved content, rather than relying on the language model's general knowledge. This ensures answers are grounded, verifiable, and free from fabricated information.

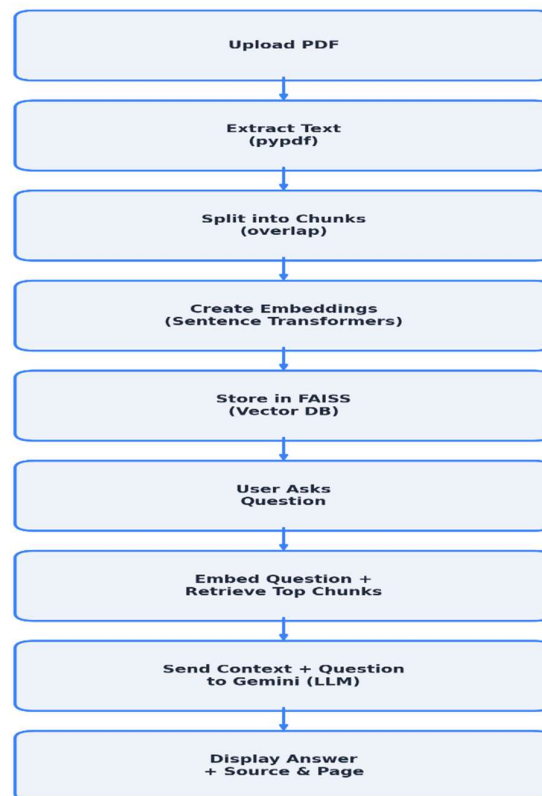
3. Tools and Technologies

Component	Tool
Programming language	Python
PDF text extraction	pypdf
Text splitting	LangChain text splitters
Embeddings	Sentence Transformers (all-MiniLM-L6-v2)
Vector database	FAISS
Language model	Google Gemini (gemini-3.6-flash), called via LangChain's langchain_google_genai integration
User interface	Streamlit
Environment variables	python-dotenv
Deployment	Streamlit Community Cloud
Version control	Git and GitHub

4. System Workflow

1. The user uploads one or more PDF files through the sidebar.
2. Each page's text is extracted using pypdf, tagged with its source filename and page number.
3. The extracted text is split into smaller overlapping chunks (chunk size ~800 characters, overlap ~120 characters) so each chunk captures a focused idea.
4. Each chunk is converted into a numerical embedding using the all-MiniLM-L6-v2 model and stored in a FAISS vector index, alongside its source and page metadata.
5. When the user asks a question, the question is embedded using the same model, and FAISS retrieves the most similar document chunks.
6. The retrieved chunks and the question are sent to Gemini with a strict system prompt instructing it to answer only from the supplied context, mention the source document and page number, and explicitly state "I could not find this information in the uploaded documents" if the answer is not present.
7. The answer is displayed in a chat interface, with the exact source document and page number shown alongside it.

Domain-Specific RAG Chatbot — Architecture



5. Core Modules

Module 1 - Document Upload: Accepts multiple PDF files, validates file type and size, displays uploaded filenames, and allows clearing or replacing documents.

Module 2 - Text Extraction: Reads every page using pypdf, preserves document name and page number as metadata, and safely skips empty or non-extractable pages.

Module 3 - Chunking: Splits extracted text into overlapping chunks using LangChain's RecursiveCharacterTextSplitter (chunk size 800, overlap 120 characters) to preserve context across chunk boundaries.

Module 4 - Embeddings and Vector Store: Converts each chunk into a numerical embedding using Sentence Transformers and stores the embeddings with metadata in a FAISS index, which can be saved and reloaded locally.

Module 5 - Retrieval: Embeds the user's question and retrieves the top 4 most similar chunks from the FAISS index, along with their source and page metadata.

Module 6 - Answer Generation: Sends the retrieved context and question to Gemini using a strict system prompt that enforces context-only answering, explicit refusal when information is unavailable, and resistance to instructions embedded within document content (prompt injection protection).

Module 7 - Streamlit Interface: Provides a PDF uploader, a Process Documents button, a chat interface with history, source citations displayed below each answer, and options to clear the chat or clear all uploaded documents.

6. Responsible AI and Security Measures

- The chatbot answers only from retrieved document context and explicitly refuses to answer when the requested information is not present, rather than generating a fabricated response.
- A visible disclaimer in the interface reminds users to verify important or high-stakes information against the original source document before relying on it.
- The system prompt explicitly instructs the model to treat any instruction-like text found within uploaded documents as ordinary content rather than as a command, guarding against prompt injection through document content.
- API keys are never committed to the repository; the .env file is excluded via .gitignore and secrets are stored securely as encrypted environment variables on Streamlit Community Cloud.
- Uploaded file types are restricted to PDF, and a file size limit is enforced to prevent processing of excessively large files.

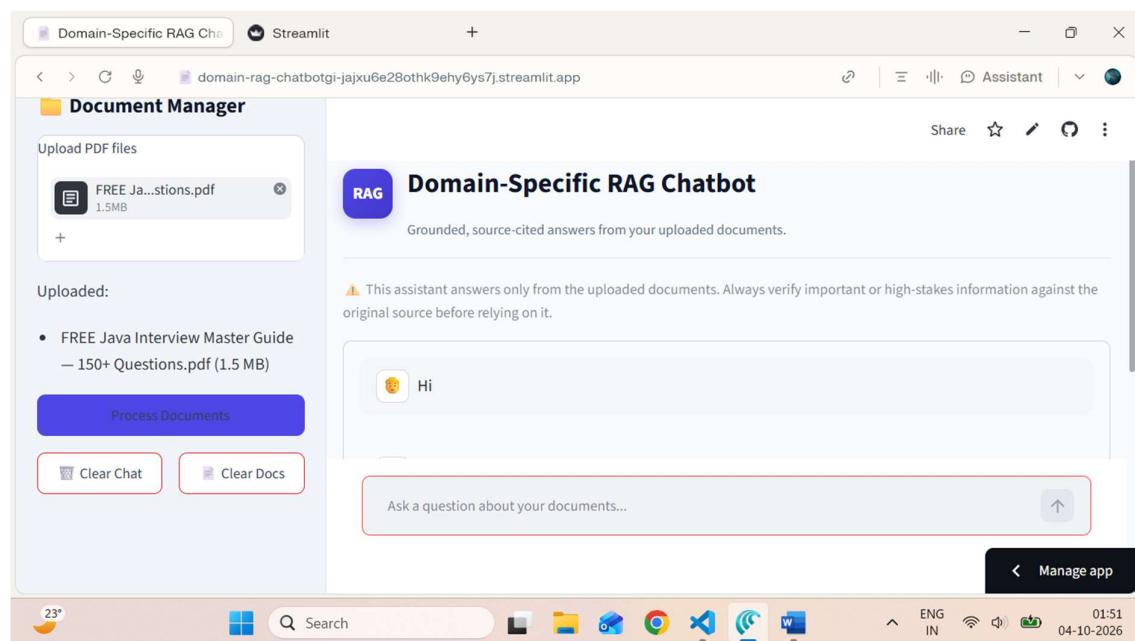
7. Testing and Evaluation

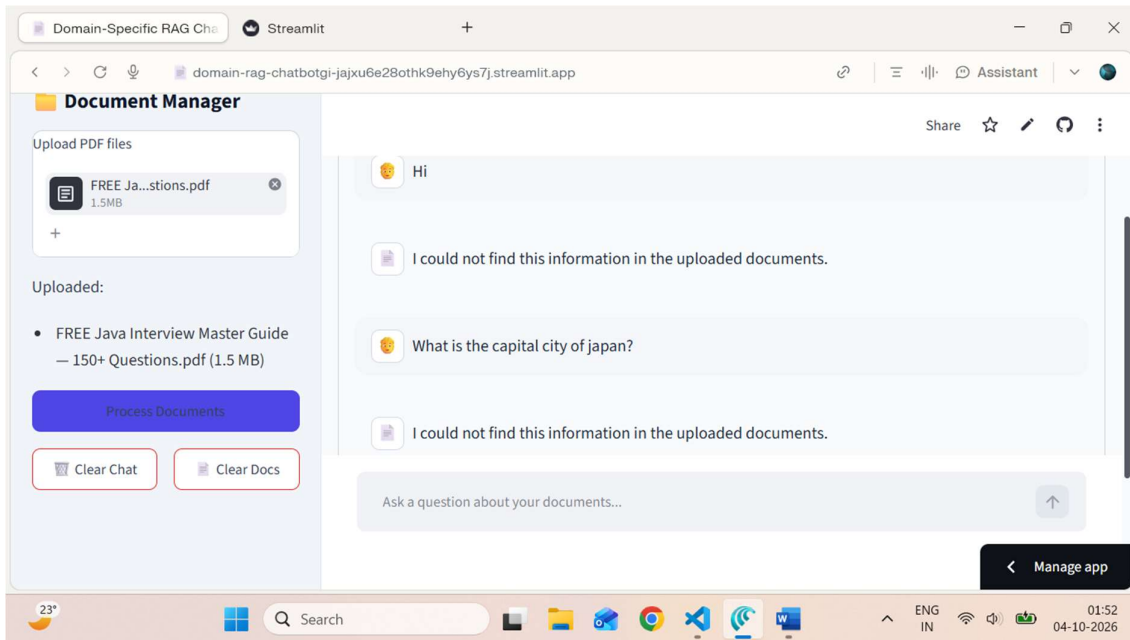
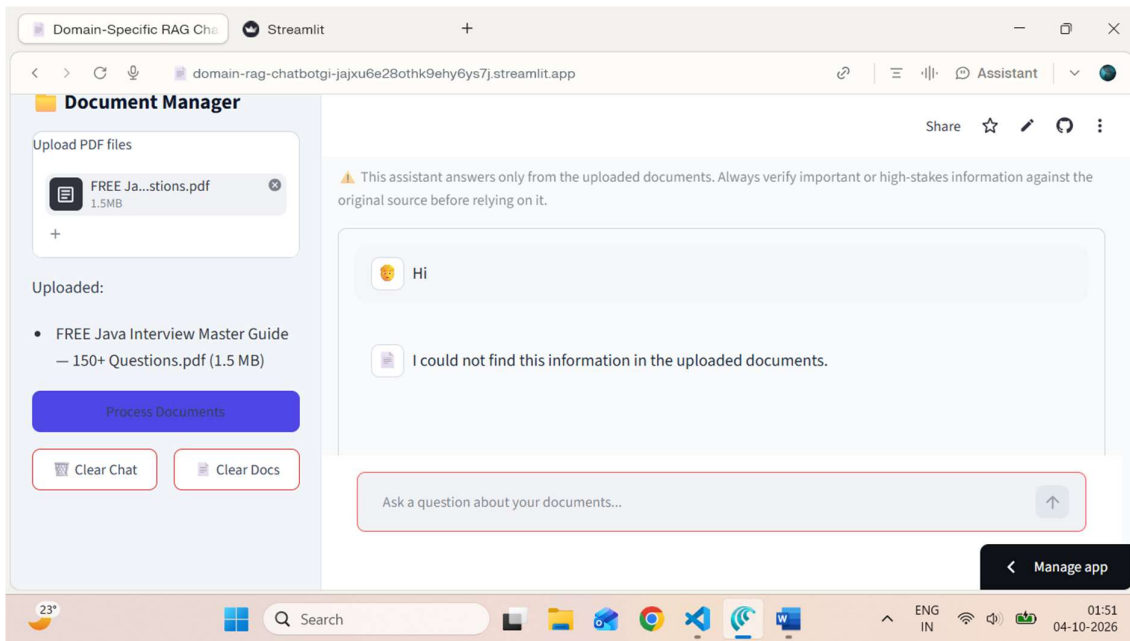
The chatbot was evaluated using 22 test questions across three categories:

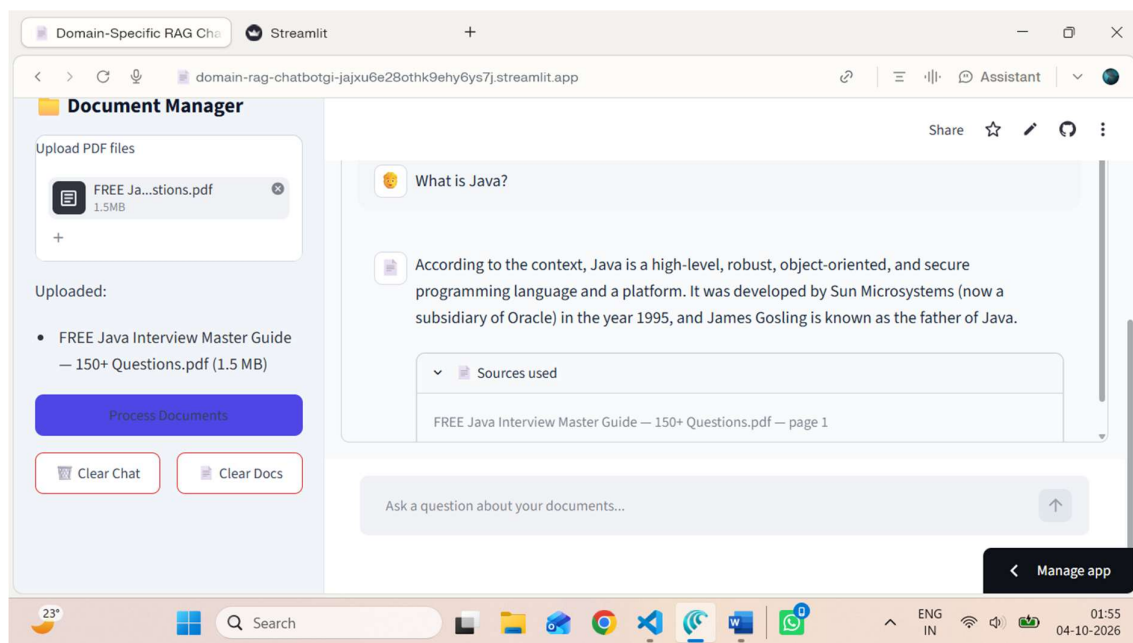
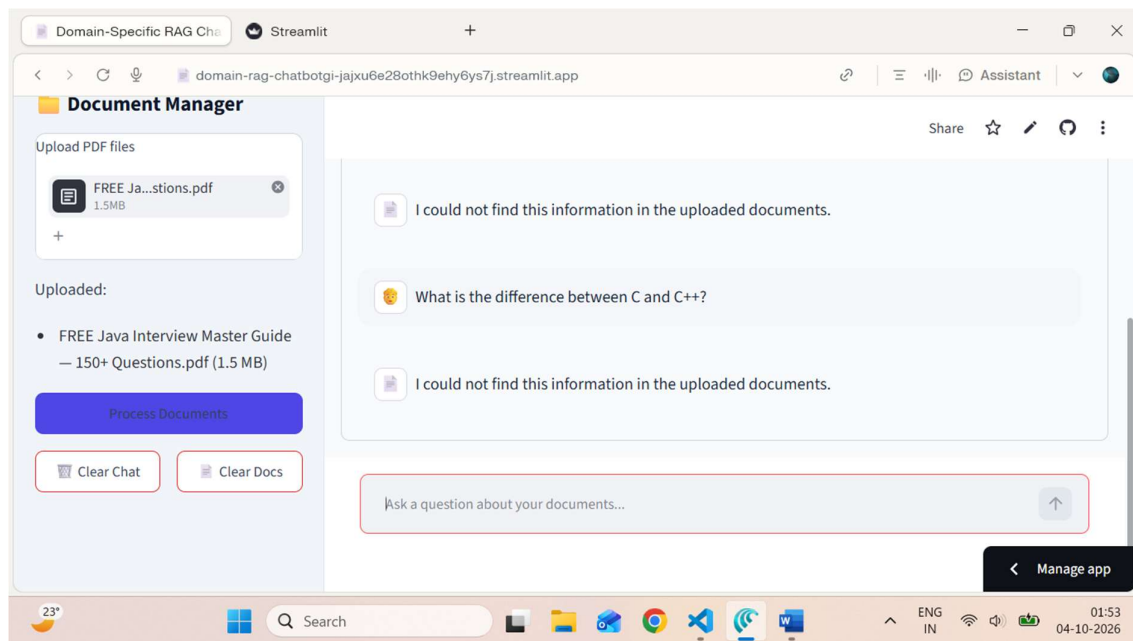
- In-scope questions (19) directly answerable from the uploaded PDFs (a Python interview Q&A guide and a Java interview master guide), verified against the correct source document and page number for each answer.
- Edge-case questions (1) a plausible-sounding, topic-adjacent question not actually covered in either document ("What is the difference between Java and JavaScript?"), used to confirm the system refuses correctly instead of generating an unsupported answer.
- Out-of-scope questions (2) questions entirely unrelated to the uploaded documents ("What is the capital of France?" and "How do you make biryani?"), used to confirm the refusal guardrail functions correctly for unrelated queries.

All 19 in-scope questions returned answers grounded in the correct source document and page number. All 3 edge-case and out-of-scope questions correctly triggered the fallback refusal message, with no hallucinated content observed in any test case.

Full question-by-question results, including expected versus retrieved source and page for each question, are available in `tests/test_questions.csv` in the project repository.







Domain-Specific RAG Chat

Streamlit

+

domain-rag-chatbotgi-jajxu6e28othk9ehy6ys7j.streamlit.app

Assistant

23°

Search

ENG IN

01:55

04-10-2026

Document Manager

Upload PDF files

FREE Ja...stions.pdf
1.5MB

+

Uploaded:

FREE Java Interview Master Guide
— 150+ Questions.pdf (1.5 MB)

Process Documents

Clear Chat

Clear Docs

Operators in Java

Based on the provided context, an operator in Java is a symbol that is used to perform operations (for example: +, -, *, / etc.).

The types of operators in Java include:

- Unary Operator
- Arithmetic Operator

Ask a question about your documents...

Manage app

Domain-Specific RAG Chat

Streamlit

+

domain-rag-chatbotgi-jajxu6e28othk9ehy6ys7j.streamlit.app

Assistant

23°

Search

ENG IN

01:56

04-10-2026

Document Manager

Upload PDF files

FREE Ja...stions.pdf
1.5MB

+

Uploaded:

FREE Java Interview Master Guide
— 150+ Questions.pdf (1.5 MB)

Process Documents

Clear Chat

Clear Docs

- Ternary Operator
- Assignment Operator

Additionally, Java includes the `instanceof` operator (also known as a type comparison operator), which is used to test whether an object is an instance of a specified type (class, subclass, or interface) and returns either `true` or `false`.

Sources used

FREE Java Interview Master Guide — 150+ Questions.pdf — page 8

Ask a question about your documents...

Manage app

8. Limitations

- Answer quality depends entirely on the content of the uploaded PDF; the chatbot cannot answer from general knowledge outside the uploaded documents.
- Scanned or image-only PDF pages without extractable text are skipped during processing.
- Retrieval occasionally misses relevant passages for loosely worded questions, correctly triggering a refusal even when related information technically exists elsewhere in the document.
- The free tier of the Gemini API enforces a daily request quota, which can interrupt testing or usage once the limit is reached.

9. Conclusion

This project demonstrates a complete, working implementation of a Retrieval-Augmented Generation pipeline for domain-specific PDF question answering. It covers the full RAG workflow text extraction, chunking, embedding, vector storage, retrieval, and grounded answer generation with a usable Streamlit interface and clear source attribution for every answer. The system was deployed to Streamlit Community Cloud and tested across in-scope, edge-case, and out-of-scope questions, confirming reliable grounding and correct refusal behavior throughout.

Links

Live App: <https://domain-rag-chatbotgi-jajxu6e28othk9ehy6ys7j.streamlit.app>

GitHub Repository: <https://github.com/Navyachalla21/domain-rag-chatbot>